Department of Computer Science and Engineering
UIET, CSJM University, Kanpur

C Programming Laboratory Assignment–5

Integer Limits, Character Codes and Format Specifications

Dr. Sandesh Gupta

The Investigator's Method

For every problem: **predict** the result, **write** the program, **compile and run** it, **record** what your computer shows, and finally **explain** whether C guarantees that result.

Quick Revision

`int`	Stores a whole number within a limited range.
`char`	Stores a character-sized integer value.
`scanf("%d", &n)`	Reads an integer into an `int`.
`scanf(" %c", &ch)`	Reads one non-whitespace character into a `char`.
`printf("%d", n)`	Displays an integer in decimal form.
`printf("%c", ch)`	Displays the character represented by a value.

Important Instructions

Work inside `~/C_Programming/Lab_05`. Use the normal `gcc source.c -o program` command taught in class. Do not copy only the output: record your prediction, actual observation and explanation separately. Some experiments intentionally contain incorrect C; run them only as directed.

Problem 1: When the Sum Is Too Large

Situation

The UIET accounts office wants to add two whole-number counts. The program works for ordinary values, but what happens when the mathematical sum is too large for an `int`?

The complete program is supplied because the `if-else` statement, `INT_MAX` and `INT_MIN` have not yet been formally taught. Create `large_sum.c` and type it exactly. Your task is to trace and understand the logic; you are not expected to write this decision independently yet.

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a, b;
    int halfSum;

    printf("Enter two integers: ");
    scanf("%d %d", &a, &b);

    halfSum = a / 2 + b / 2;

    if (halfSum >= INT_MIN / 2 &&
        halfSum <= INT_MAX / 2)
    {
        printf("Sum = %d\n", a + b);
    }
    else
    {
        printf("Sum is outside the range of int.\n");
    }

    return 0;
```

© 2026 Sandesh Gupta

```
}
```

Understanding the Half-Sum Logic

halfSum is deliberately declared as an **int**. Therefore **a / 2** and **b / 2** use integer division. A possible .5 part is ignored in each division. The algorithm compares this smaller result with half of the permitted integer range. If the comparison appears safe, it prints **a + b**; otherwise, it prints an out-of-range message.

Three New Things Used in the Supplied Program

- **#include <limits.h>** provides **INT_MAX** and **INT_MIN**, the largest and smallest values supported by **int** on the current system.
- **if (condition)** runs its first block when the condition is true; **else** runs the other block when it is false. Here, **&&** means that both range comparisons must be true.

Run the program with these cases:

First	Second	Observed output
20	30	
1000000	2000000	
2147483646	1	
2147483647	1	
-2147483648	-1	
2147483647	-2147483648	

© 2026 SANDESH GUPTA UIET CSJM UNIVERSITY, KANPUR

Answer these questions:

1. Which tests are rejected by the program, and why?
2. Trace **halfSum** for **a = 2147483647** and **b = 1**.
3. Which two .5 portions are lost during that calculation?
4. Does the condition identify this pair as safe or unsafe? Is that decision actually correct?
5. Suggest how a future version might include the discarded remainders.

Intentional Limitation—Correct It Later

The simplified algorithm ignores the .5 discarded by integer division. Consequently, an exact boundary such as **INT_MAX + 1** may be incorrectly classified as safe. Evaluating that addition causes signed integer overflow, for which C guarantees no particular result.
For this assignment, observe and explain the limitation. After learning more control statements and data types, improve the algorithm so that it also uses the remainders and never evaluates an overflowing **int** addition.

The Main Lesson: Logic Before Language

Knowing the syntax of **+** and **if** is not enough. The programmer must design and test the algorithm, especially at boundary values. This example shows both a limitation of the **int** type and a limitation of an apparently reasonable first algorithm.

© 2026 Sandesh Gupta

Problem 2: A Character Written into an Integer

Situation

An integer occupies several bytes, while **%c** is meant to read a character. What happens if we intentionally give **scanf** the address of an integer?

Create **character_into_integer.c** exactly as shown:

```c
#include <stdio.h>

int main(void)
{
    int number = 256;

    printf("Enter one character: ");
    scanf("%c", &number);
    printf("Integer value = %d\n", number);

    return 0;
}
```

Before running it, write your prediction for input **A**:

- Will the output be 65, the commonly used ASCII value of **A**?
- Will the original value 256 have any effect?
- Could the result be 321 on a commonly used little-endian computer?

Compile normally and record any message produced by the compiler. Run the program, enter **A**, and record the observed integer. Repeat after changing the initialization from 256 to 0. Do the two observations differ?

This Is an Intentional Type Mismatch

For **%c**, **scanf** requires the address of a **char**, but **&number** is the address of an **int**. Therefore the program has **undefined behaviour**; no output is guaranteed. On one common little-endian, ASCII-based system, entering **A** after initializing the integer to 256 may appear to produce 321 because only one byte may be changed. Another system may behave differently.

In your notebook, draw four boxes for the four bytes of a typical **int**. Use them to explain why initializing the integer with 0 and 256 can lead to different observed values even when the entered character is the same.

Problem 3: Is the Character Digit '7' the Number 7?

Situation

A student types the key **7**. The keyboard supplies a character. Does printing that character with **%d** produce the numerical value 7?

Create **digit_value.c** using the correct variable and matching format:

```c
#include <stdio.h>

int main(void)
{
    char digit;

    printf("Enter one digit character: ");
    scanf(" %c", &digit);

    printf("As a character = %c\n", digit);
    printf("Character code = %d\n", digit);
    printf("Numeric value  = %d\n", digit - '0');
```

© 2026 Sandesh Gupta

```
    return 0;
}
```

Predict and test the inputs `0`, `5` and `7`. Complete this table:

Input character	Printed character code	Result of `digit - '0'`
0		
5		
7		

Answer these questions:

1. On the UIET lab computer, what code is displayed for `'7'`?
2. Why is the code normally 55 rather than 7 on an ASCII-based system?
3. Why does `digit - '0'` give the desired numerical digit value?
4. What result do you observe if the input is `A`? Is that input a decimal digit, and should its subtraction result be treated as one?

Important Difference

`'7'` is a character value; 7 is an integer value. C guarantees that the codes for `'0'` through `'9'` are consecutive, so subtracting `'0'` converts a valid digit character to its numerical value. The exact character code is system-dependent, although ASCII is commonly used.

Final Comparison and Demonstration

Complete the statements using **guaranteed**, **commonly observed**, or **undefined**:

1. The result of `20 + 30` in an ordinary `int`: _____
2. The result of signed integer overflow: _____
3. Reading `%c` into an `int *`: _____
4. The fact that digit character codes are consecutive: _____
5. The assumption that every C system uses ASCII: _____

Show Before Leaving the Lab

Show the instructor all three source files, your prediction tables and the actual outputs. Explain why a program can compile and run yet still have no guaranteed result. Demonstrate the difference between character `'7'` and integer `7`.

A surprising output is an observation; the C rules decide whether it is reliable.

© 2026 Sandesh Gupta